

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	Kanmani L
Name	192511414

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Kanmani L/192511414 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

Answer: HDFS-Based Storage Solution for a Media Platform

Problem:

A media platform stores a large volume of videos, images, and audio files. The system must maintain more than 99.9% availability, use a maximum replication factor of 3, and remain cost-efficient as storage requirements increase.

Proposed Design:

Use an HDFS cluster with an Active NameNode, Standby NameNode, and multiple DataNodes. Set the replication factor to 3 and use rack-aware replica placement.

Replication:

Each HDFS block is stored on three different DataNodes. If one DataNode fails, the remaining replicas can still provide the data. Rack-aware placement ensures that replicas are distributed across different racks to protect against rack-level failures.

High Availability:

Configure HDFS High Availability using Active and Standby NameNodes. If the Active NameNode fails, the Standby NameNode takes over, reducing service interruption and supporting the required availability.

Cost-Efficient Storage:

Use large HDFS block sizes, such as 256 MB, for large media files to reduce metadata overhead. Instead of purchasing excessive storage initially, new DataNodes can be added gradually as the amount of media data grows. HDFS rebalancing can distribute data across the newly added nodes.

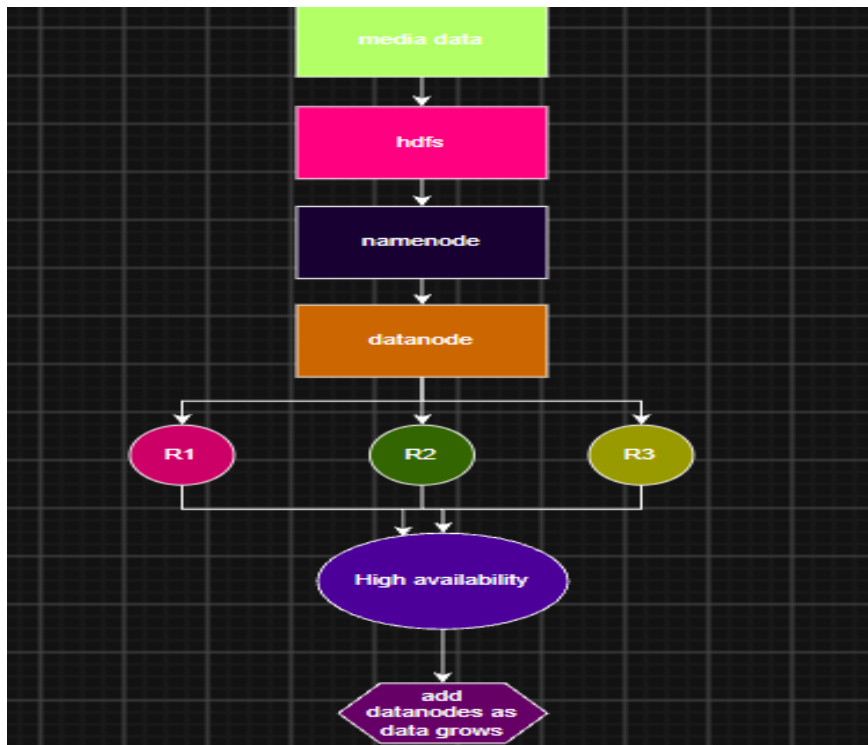
Fault Tolerance:

HDFS automatically detects failed DataNodes and creates replacement replicas when necessary. This maintains the required level of data protection without exceeding the replication factor of 3.

Constraint Satisfaction:

Constraint	Solution
Availability >99.9%	NameNode HA and 3 replicas
Replication ≤ 3	Replication factor = 3
Hardware failure	Rack-aware replication
Cost efficiency	Incremental DataNode addition
Large media files	Large HDFS block size

The proposed HDFS solution provides high availability and fault tolerance through three-way replication and NameNode HA. Rack-aware storage protects against rack failures, while incremental cluster expansion keeps the solution cost-efficient as media data continuously grows.



2.A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

Answer:

MapReduce Workflow for Log Processing Within 20 Minutes

Problem:

A large amount of log data must be processed within a 20-minute batch window using a limited number of cluster nodes. The job must therefore maximize parallelism while minimizing network and processing overhead.

Proposed Design:

Store the logs in HDFS and divide them into blocks. Multiple Mapper tasks process these blocks in parallel. A Combiner is used to perform local aggregation before the Shuffle phase, followed by balanced Reducer tasks.

Map Phase:

Each Mapper reads log records and extracts required information such as IP address, URL, status code, number of requests, or response time. Since HDFS blocks are distributed across nodes, Mapper tasks should preferably execute on the nodes containing the required data. This provides data locality and reduces network traffic.

Combiner:

The Combiner performs local aggregation of Mapper output before transferring it to Reducers. For example, several occurrences of an HTTP 500 error can be combined into a single count. This reduces the amount of intermediate data transferred during the Shuffle phase.

Shuffle and Sort:

The intermediate key-value pairs are partitioned and transferred to appropriate Reducers. Proper partitioning ensures that the workload is distributed evenly and prevents one Reducer from becoming a bottleneck.

Reduce Phase:

Reducers aggregate the received data and generate the final log-processing results. The number of Reducers should be selected according to the available CPU, memory, input size, and required processing time.

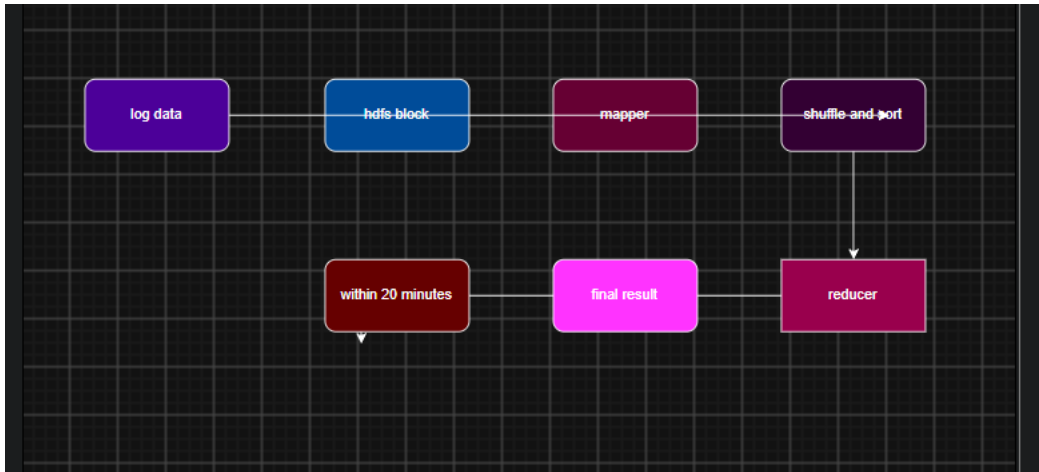
Techniques to Meet the 20-Minute Constraint:

- Execute Mapper tasks in parallel.
- Use data locality to reduce network transfer.
- Use a Combiner to reduce intermediate data.
- Balance data among Reducers.
- Avoid unnecessary transformations and disk operations.
- Use speculative execution when a task is significantly slower than others.
- Allocate available cluster resources efficiently.

Constraint Satisfaction:

Constraint	Solution
20-minute deadline	Parallel MapReduce execution
Limited nodes	Efficient task distribution
Large log data	HDFS block-based processing
Network overhead	Combiner and data locality
Uneven workload	Balanced partitioning
Slow tasks	Speculative execution

The proposed MapReduce workflow uses parallel processing, data locality, combiners, and balanced reducers to efficiently process log data with limited cluster resources. These optimizations reduce processing and communication overhead and help the job complete within the required 20-minute window.



3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

Answer:

YARN Scheduler Policy for ETL, Reporting and Machine Learning

Problem:

A YARN cluster supports ETL, reporting, and machine learning workloads. The scheduling policy must provide fairness, give higher priority to ETL, and recover from node failures.

Proposed Design:

Use the YARN Capacity Scheduler with separate queues for ETL, reporting, and machine learning jobs. ETL is assigned the highest priority because downstream applications may depend on the availability of processed data.

Queue Allocation:

A suitable initial allocation could be 50% for ETL, 25% for reporting, and 25% for machine learning. These values can be adjusted based on workload requirements.

ETL Priority:

ETL jobs receive higher scheduling priority because they are often part of critical data pipelines. However, capacity limits prevent ETL from consuming the entire cluster and ensure that reporting and ML jobs continue to receive resources.

Fairness:

Fairness is maintained using queue capacity limits, user limits, and resource quotas. Multiple jobs within the same queue should share the available resources rather than allowing a single application to consume all resources.

Resource Management:

YARN's ResourceManager manages the overall cluster resources, while NodeManagers manage resources on individual worker nodes. Applications receive containers based on queue capacity, priority, and available resources.

Node Failure Recovery:

If a NodeManager or worker node fails, the ResourceManager detects the failure. Containers running on the failed node can be restarted on healthy nodes. Failed tasks are therefore rescheduled without requiring the entire application to restart.

Constraint Satisfaction:

Requirement	Mechanism
ETL priority	High-priority ETL queue
Fairness	Queue capacity and user limits
Reporting	Dedicated reporting queue
ML workloads	Dedicated ML queue
Node failure	Container/task rescheduling
Resource efficiency	Dynamic resource allocation

A YARN Capacity Scheduler with dedicated queues provides high priority to ETL while maintaining fairness for reporting and machine learning workloads. Resource limits prevent starvation, while YARN's failure detection and container rescheduling provide recovery from node failures.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

Answer:

Enterprise Data Ingestion from SAN, NAS and Operational Databases

Problem:

An enterprise needs to ingest data from SAN, NAS, and operational databases into a big-data platform. The architecture must provide reliable backup while minimizing unnecessary data duplication.

Proposed Architecture:

Use Apache NiFi as the centralized ingestion layer and HDFS as the distributed storage layer. NiFi collects data from different sources, performs required transformations and filtering, and transfers the data into HDFS.

SAN Integration:

Data stored in SAN can be accessed through suitable enterprise storage interfaces and transferred through the ingestion pipeline. Only required datasets should be ingested to avoid unnecessary copies.

NAS Integration:

NAS generally contains file-based data. NiFi can monitor selected directories and automatically transfer newly created or modified files into HDFS.

Operational Database Integration:

NiFi can connect to operational databases using JDBC-based processors. Incremental data extraction should be used instead of repeatedly copying the complete database.

For example, after processing records up to ID 10,000, the next extraction can retrieve only records with IDs greater than 10,000.

Data Deduplication:

Duplication can be minimized using unique record IDs, timestamps, database checkpoints, file hashes, and incremental extraction. NiFi's data provenance can also help track previously processed data.

HDFS Storage:

HDFS provides distributed and fault-tolerant storage. Raw and processed data should be maintained separately so that the original data remains available for recovery and reprocessing.

Backup Reliability:

HDFS replication protects against individual node failures. For stronger backup protection, critical data should also be backed up to independent storage. Important metadata and recovery information should be included in the backup strategy.

Monitoring and Error Handling:

NiFi provides monitoring, provenance tracking, back-pressure, routing, and failure handling. Failed transfers can be routed to separate error flows and retried without repeating successful transfers.

Constraint Satisfaction:

Constraint	Solution
SAN ingestion	Controlled storage ingestion
NAS ingestion	NiFi file-based ingestion
Database ingestion	JDBC incremental extraction
Minimal duplication	IDs, checkpoints and hashes
Reliable storage	HDFS replication
Backup	Independent backup storage

Monitoring	NiFi provenance and error handling
------------	------------------------------------

Conclusion:

The proposed architecture uses Apache NiFi for controlled ingestion and HDFS for distributed storage. Incremental extraction, checkpoints, and unique identifiers minimize duplication, while HDFS replication and independent backups provide reliable data protection.

5.Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Answer:

End-to-End Hadoop Ecosystem with Secure Data Movement and High Availability

Problem:

An organization requires an end-to-end Hadoop ecosystem that supports secure data movement, high availability, and integration with Apache NiFi or ETL pipelines.

Proposed Design:

The architecture consists of data sources, Apache NiFi, HDFS, YARN, and processing tools such as MapReduce, Spark, and Hive.

Data Ingestion and ETL:

Apache NiFi acts as the central ingestion and ETL layer. It collects data from databases, files, applications, and other sources. NiFi can perform data validation, filtering, transformation, routing, and deduplication before storing the data in HDFS.

Secure Data Movement:

Data should be protected during transfer using TLS/SSL encryption. Hadoop components can use Kerberos authentication to verify users and services. Access permissions should be configured so that only authorized users and applications can access sensitive data.

HDFS High Availability:

Configure HDFS with an Active NameNode and Standby NameNode. Multiple DataNodes store the actual data blocks. A replication factor of 3 can be used to protect data from DataNode failures.

If the Active NameNode fails, the Standby NameNode takes over. If a DataNode fails, another replica remains available and HDFS can create a replacement replica.

YARN Resource Management:

YARN manages cluster resources such as CPU and memory. It allocates containers to applications including MapReduce, Spark, and Hive. This allows multiple workloads to execute efficiently on the same Hadoop cluster.

Data Processing:

MapReduce can be used for large batch-processing workloads, Spark can be used for fast and iterative processing, and Hive can provide SQL-based analysis of data stored in HDFS.

ETL Pipeline:

The overall data flow is:

Extract → NiFi Ingestion → Validate → Transform → Deduplicate → HDFS → Process using MapReduce/Spark/Hive → Analytics

This provides a controlled path from data sources to final analytics.

Access Control and Security:

HDFS permissions and role-based access controls restrict access to sensitive datasets. Encryption can be applied to data at rest where required, while TLS protects data in transit. Audit logs can be maintained to track access and security-related activities.

Failure Recovery:

HDFS replication protects data from DataNode failures. NameNode HA protects the metadata service. YARN can reschedule failed containers on healthy nodes. NiFi can retry failed data transfers and route failed records for further investigation.

Monitoring:

The system should continuously monitor NameNode and DataNode health, HDFS storage, YARN resources, NiFi queues, failed pipelines, and security events.

Constraint Satisfaction:

Constraint	Solution
Secure data movement	TLS/SSL
Authentication	Kerberos
High availability	Active + Standby NameNode
Data protection	HDFS replication
Resource management	YARN
ETL integration	Apache NiFi
Batch processing	MapReduce
Fast processing	Spark
SQL analytics	Hive
Failure recovery	HDFS and YARN recovery

The proposed Hadoop ecosystem provides a secure, highly available, and scalable big-data platform. Apache NiFi manages secure data ingestion and ETL, HDFS provides fault-tolerant storage, YARN manages resources, and MapReduce, Spark, and Hive support different processing requirements. Thus, the design satisfies the constraints of secure data movement, high availability, and ETL integration.

